Höhere technische
Bundeslehranstalt
und Bundesfachschule
im Hermann Fuchs
Bundesschulzentrum

Ascenta - Guiding your Vision

Diplomarbeit

Schulautonomer Schwerpunkt
Coding & AI

ausgeführt im Schuljahr 2025/2026 von:

Liam Vendel, 5AHELS

Manuel Zauner, 5AHELS

Fabian Lindlbauer, 5AHELS

Betreuer:

Dipl.-Ing. Mario Preishuber

Braunau am Inn, am 11.02.2026

Eidesstattliche Erklärung

Es wird an Eides statt erklärt, dass die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als angegebene Quellen und Hilfsmittel nicht direkt benutzt und die benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht wurden.

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Liam Vendel

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Manuel Zauner

Braunau, 11.02.2026

. . . . . .

Ort und Datum

. . . . . .

Fabian Lindlbauer

Abstract - Liam

This diploma thesis details the development of “Ascenta”, an intelligent assistive wearable designed to support visually impaired individuals. Given the challenges of navigating complex urban environments, the project aims to create an affordable solution using artificial intelligence to provide information about the user’s social surroundings.

The technical framework utilizes the Arduino Nicla Vision microcontroller. The firmware implements the FOMO architecture for efficient on-device face detection. For user interaction, a local and privacy-oriented voice recognition system was developed using the **Vosk engine** within a custom Android application. Communication is established via an optimized TCP protocol to ensure low-latency data streaming between the wearable and the mobile device.

The current implementation successfully recognizes faces within the field of view and determines proximity using Time-of-Flight sensors. The results demonstrate the efficiency of the decentralized architecture and provide a solid foundation for future development of general obstacle detection models.

Kurzfassung - Liam

Die vorliegende Diplomarbeit dokumentiert die Entwicklung von „Ascenta“, einer intelligenten Assistenzbrille zur Unterstützung sehbeeinträchtigter Personen im Alltag. Angesichts der Komplexität moderner urbaner Umgebungen besteht das Ziel des Projekts in der Realisierung eines kostengünstigen Wearables, das mittels künstlicher Intelligenz Informationen über das soziale Umfeld liefert.

Die technische Umsetzung basiert auf dem Arduino Nicla Vision Mikrocontroller. Die Firmware nutzt die FOMO-Architektur (Faster Objects, More Objects) zur effizienten On-Device-Gesichtserkennung. Zur Interaktion wurde eine lokale, datenschutzkonforme Sprachverarbeitung auf Basis der **Vosk-Engine** in einer Android-Applikation implementiert. Die Kommunikation erfolgt über ein optimiertes TCP-Protokoll, um minimale Latenzzeiten beim Datenstreaming zwischen der Brille und dem Smartphone zu gewährleisten.

Das realisierte System ist in der Lage, Personen im Sichtfeld zu identifizieren und die Distanz mittels Time-of-Flight-Sensorik zu bestimmen. Die Ergebnisse belegen die Funktionalität der dezentralen Architektur und bilden die Grundlage für zukünftige Erweiterungen zur allgemeinen Hinderniserkennung.

Contents

Abstract iii

Kurzfassung iv

1 Einleitung - Liam 2
- 1.1 Motivation und Ausgangssituation . . . 2
- 1.2 Problemstellung . . . 2
- 1.3 Zielsetzung . . . 2
- 1.4 Lösungsansatz und Kernidee . . . 3

2 Szenarien und Use Cases - Liam 4
- 2.1 Use Case A: Kollisionsvermeidung mit Personen . . . 4
  - 2.1.1 Ausgangssituation . . . 4
  - 2.1.2 Systemverhalten . . . 4
- 2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl . . . 5
  - 2.2.1 Ausgangssituation . . . 5
  - 2.2.2 Systemverhalten . . . 5

3 Konzept 6
- 3.1 Gesamtüberblick . . . 6
- 3.2 Komponenten . . . 6
  - 3.2.1 Mechanik und Gehäusedesign . . . 6
  - 3.2.2 Hardware und Elektronik . . . 6
  - 3.2.3 Software-Ökosystem und Netzwerkkonzept . . . 7
- 3.3 Interaktionen . . . 8
- 3.4 Technisches Design . . . 8
  - 3.4.1 Lokale Datenverarbeitung und Souveränität . . . 9
  - 3.4.2 Absicherung der Netzwerkschnittstellen . . . 9

4 Umsetzung 10
- 4.1 Implementierung der Komponenten . . . 10
  - 4.1.1 Software-Architektur und Datenverarbeitung - Liam . . . 10
  - 4.1.2 Schaltungsdesign und Energiemanagement . . . 23
  - 4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung) . . . 23

4.1.4 Konstruktion und 3D-Druck 23
4.2 Herausforderungen in der Entwicklung 23
4.2.1 Mechanische und geometrische Restriktionen 23
4.2.2 Energiemanagement im begrenzten Bauraum 23
4.2.3 Ressourcenlimits und Speicher-Fragmentierung (Software) - Liam 23
4.3 Tests und Validierung - Liam 24
4.3.1 Netzwerklatenz und Streaming-Performance 24
4.3.2 Auswertung der kinetischen Interaktion (Tap-Detection) 24
4.3.3 Energetische Evaluierung (Akkulaufzeit) 25

5 Ausblick (Future Work) 26
5.1 Software und Benutzeroberfläche - Liam 26
5.1.1 Native Zero-UI-Applikation für blinde Nutzer 26
5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip) . 26
5.2 Künstliche Intelligenz und Modell-Training 27
5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände 27
5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse) 27
5.3 Hardware und Elektronik 27
5.3.1 Optimierung der Schaltungstechnik und Sensorik 27
5.4 Mechanik und Gehäusedesign 28
5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie 28

6 Verwandte Arbeiten (Related Work) - Liam 29
6.1 Klassische Mobilitätshilfen 29
6.2 Smartphone-basierte Applikationen 29
6.3 Kommerzielle Smart Glasses und Wearables 30
6.4 Positionierung des von Ascenta 30

7 Zusammenfassung und Ausblick - Liam 31

8 Project Management 32
8.1 Planung und Organisation 32
8.2 Evaluierung der Arbeitsstunden 32
8.3 Zeitplan 33
8.3.1 Liam Vendel 33
8.3.2 Manuel Zauner 33
8.3.3 Fabian Lindlbauer 33

Acknowledgements 35

Abbildungsverzeichnis 36

Quellcodeverzeichnis 36

Literaturverzeichnis 37

CV 39

1 Einleitung - Liam

1.1 Motivation und Ausgangssituation

Stellen Sie sich vor, Sie navigieren während der Stoßzeit durch einen belebten Bahnhof. Sie verlassen sich auf Schilder, um den Ausgang zu finden, prüfen Abfahrtstafeln und achten auf andere Pendler. Für weltweit etwa 285 Millionen sehbehinderte Menschen stellt dieses Szenario eine tägliche Herausforderung voller Barrieren dar. Während der klassische Langstock Hindernisse am Boden erkennt, versagt er bei der Identifikation von Personen oder Objekten auf Kopfhöhe. Bestehende elektronische Hilfsmittel sind oft aufgrund extrem hoher Kosten ($>$ 2.500 Euro) oder einer Abhängigkeit von permanenter Cloud-Konnektivität für die breite Masse unzugänglich.

1.2 Problemstellung

Das zentrale Problem liegt in der mangelnden digitalen Inklusion aufgrund finanzieller und technologischer Hürden. Mobile Assistenzsysteme müssen eine Echtzeit-Verarbeitung sensibler Bild- und Audiodaten ermöglichen, ohne die Ergonomie des Nutzers durch hohes Gewicht oder Wärmeentwicklung zu beeinträchtigen. Zudem erfordern Cloud-basierte Ansätze eine ständige Internetverbindung, was die Zuverlässigkeit in geschlossenen Räumen (z. B. U-Bahnhöfen) mindert und den Datenschutz gefährdet.

1.3 Zielsetzung

Das Ziel des Projekts „Ascenta“ ist die Entwicklung eines Prototyps einer smarten Assistenzbrille, die visuelle Umweltinformationen in akustisches Feedback übersetzt. Das System soll kostengünstig (Materialkosten $<$ 150 Euro) realisiert werden, eine diskrete Bauweise aufweisen und durch lokale Datenverarbeitung (Offline-Betrieb) maximale Datensicherheit für den Nutzer garantieren.

1.4 Lösungsansatz und Kernidee

Die Lösung basiert auf einer dezentralen „Split-Brain“-Architektur. Die Brille (Arduino Nicla Vision) fungiert als smarter Sensor-Satellit und führt eine ressourcensparende Objekterkennung (FOMO) zur Gesichtserkennung aus. Die rechenintensive Logik sowie die Spracherkennung werden auf das Smartphone des Nutzers ausgelagert. Die Übertragung erfolgt über ein latenzarmes WiFi-TCP-Protokoll. Durch diesen Ansatz wird die Hardware an der Brille entlastet, während die volle Prozessorleistung moderner Smartphones für die Sprachverarbeitung mittels Vosk genutzt wird.

2 Szenarien und Use Cases - Liam

Um die funktionalen Anforderungen an das Assistenzsystem „Ascenta“ zu definieren und die spätere technische Umsetzung zu rechtfertigen, werden im Folgenden konkrete Anwendungsszenarien (Use Cases) beschrieben. Diese Szenarien repräsentieren alltägliche Herausforderungen von stark sehbeeinträchtigten oder blinden Personen, die durch das System gelöst werden. Gemäß dem aktuellen Entwicklungsstand fokussieren sich die Anwendungsfälle auf die Erkennung von Personen im direkten Umfeld, die Distanzabschätzung sowie die auditive Interaktion.

2.1 Use Case A: Kollisionsvermeidung mit Personen

2.1.1 Ausgangssituation

Ein Nutzer bewegt sich mit einem regulären Langstock durch einen öffentlichen Raum, beispielsweise ein Foyer oder einen Gehweg. Während der Langstock Bodenhindernisse (wie Treppenstufen oder Bordsteinkanten) zuverlässig ertastet, registriert er keine stillstehenden oder sich nähernden Personen auf Körper- oder Kopfhöhe, bevor es zu einem physischen Kontakt kommt.

2.1.2 Systemverhalten

Das Kamerasystem der Ascenta-Brille erfasst kontinuierlich das Sichtfeld des Nutzers. Sobald eine Person in den Erfassungsbereich tritt, wird diese durch das lokale Machine-Learning-Modell (FOMO) auf dem Mikrocontroller identifiziert. Simultan misst der Time-of-Flight-Sensor (ToF) die exakte Distanz zu diesem Objekt.

Unterschreitet die Person eine vordefinierte kritische Distanz (beispielsweise einen Meter), fusioniert die Smartphone-Applikation diese Sensordaten und generiert unmittelbar ein akustisches Warnsignal. Der Nutzer wird dadurch in Echtzeit auf das Hindernis aufmerksam gemacht und kann rechtzeitig ausweichen, ohne dass ein physischer Kontakt entsteht.

2.2 Use Case B: Kontextbezogene Umgebungsabfrage per Sprachbefehl

2.2.1 Ausgangssituation

Der Nutzer befindet sich in einer Warteschlange oder an einem Haltepunkt (z. B. einer Bushaltestelle) und möchte wissen, ob sich eine Person direkt vor ihm befindet, um entsprechend aufrücken oder Abstand halten zu können. Ein weites Ausschwenken des Langstocks ist in dieser Situation unerwünscht, da es andere Personen stören oder irritieren könnte.

2.2.2 Systemverhalten

Anstatt die Umgebung physisch abzutasten, nutzt der Anwender die integrierte Sprachsteuerung. Der Nutzer aktiviert das System (beispielsweise durch ein definiertes Interaktionsmuster an der Brille) und stellt verbal eine Frage, wie etwa: „Befindet sich jemand vor mir?“.

Das Audiosignal wird über das in der Brille verbaute Mikrofon erfasst und via TCP-Protokoll latenzarm an das verbundene Smartphone gestreamt. Dort verarbeitet die lokale Vosk-Engine den Sprachbefehl offline, um den Datenschutz zu gewährleisten. Die Applikation gleicht die Anfrage mit den aktuellen Kameradaten (Personenerkennung) und der Distanzmessung (ToF) ab. Anschließend erhält der Nutzer über die Sprachausgabe (Text-to-Speech) ein diskretes, akustisches Feedback, beispielsweise: „Person in einem Punkt fünf Metern Entfernung erkannt.“ Dadurch gewinnt der Nutzer ohne stigmatisierende Bewegungen ein präzises Verständnis seiner unmittelbaren sozialen Umgebung.

3 Konzept

Dieses Kapitel beschreibt die abstrakte Architektur des Assistenzsystems „Ascenta“. Es liefert einen Gesamtüberblick über das funktionale Zusammenwirken der Teilsysteme und erläutert die iterativen Designentscheidungen, welche zur finalen Netzwerktopologie führten.

3.1 Gesamtüberblick

Das System basiert auf einer dezentralen „Split-Brain“-Architektur. Um die Hardware direkt am Kopf des Nutzers so leicht und unauffällig wie möglich zu gestalten, lagert die Architektur die ressourcenintensive Datenverarbeitung auf das Smartphone des Nutzers aus. Die Brille fungiert als reiner Sensor- und Kommunikationsknotenpunkt (Edge-Device). Sie erfasst Umgebungsdaten und streamt die Ergebnisse einer lokalen Vorverarbeitung an das mobile Endgerät. Das Smartphone übernimmt die Rolle des Hauptrechenzentrums. Es fusioniert die Sensordaten, interpretiert Benutzereingaben und generiert das akustische Feedback.

3.2 Komponenten

Das Gesamtsystem setzt sich aus drei wesentlichen logischen Bausteinen zusammen: der physischen Struktur (Gehäuse), der elektronischen Hardware (Sensorik und Energieversorgung) und dem Software-Ökosystem.

3.2.1 Mechanik und Gehäusedesign

Manuel

3.2.2 Hardware und Elektronik

Fabian

3.2.3 Software-Ökosystem und Netzwerkkonzept

Die Softwarearchitektur teilt sich in die Firmware des Mikrocontrollers und die mobile Applikation. Die Firmware (MicroPython) verantwortet die hardwarenahe Sensorsteuerung und eine erste Datenreduktion (Edge AI mittels FOMO). Die Android-Applikation (Kotlin) dient als unsichtbares Interface (Zero-UI) und integriert komplexe Algorithmen wie die Offline-Spracherkennung (Vosk) zur auditiven Interaktion.

Evolution der Netzwerkarchitektur (BLE → UDP → TCP)

Die finale Netzwerktopologie resultiert aus einer dreistufigen Evaluierungsphase. Da das System hohe Anforderungen an Latenz und Datensicherheit stellt, dokumentiert der folgende Prozess die Abwägung zwischen verschiedenen Protokollstacks.

Phase 1: Das initiale BLE-GATT-Profil
Das ursprüngliche Konzept sah das Nicla Vision als eigenständiges BLE-Peripheral vor. Das System spannte einen GATT-Server auf, um Daten über spezifische Characteristics zu übertragen. Die Übertragung von Audiodaten (ca. 128.000 Bytes) stieß jedoch auf massive Barrieren. BLE-Pakete sind im Standard auf eine Payload (MTU) von 20 Bytes limitiert. Um den Hardware-Puffer des Mikrocontrollers zu schützen und einen `OSError 11` zu vermeiden, musste der Versand künstlich auf 15 ms Intervalle gedrosselt werden. Dies führte zu inakzeptablen Latenzen von mehreren Sekunden für eine einzige Sprachnachricht.

```
# 16000 Hz * 1 Channel * 2 Bytes * 4.0s = 128.000 Bytes Pufferbedarf
TOTAL_BYTES = int(SAMPLE_RATE * CHANNELS * BYTES_PER_SAMPLE *
    RECORD_SECONDS)
UART_SERVICE_UUID = bluetooth.UUID("6E400001-B5A3-F393-E0A9-E50E24DCCA9E
    ")
```

Listing 3.1: Berechnung des Audiospeichers und initiales BLE-Setup

Phase 2: Der Zwischenschritt über das User Datagram Protocol (UDP)
Um die BLE-Limitierung zu umgehen, erfolgte der Wechsel auf ein WiFi-basiertes Netzwerk unter Nutzung von UDP. Dieses Protokoll ermöglichte theoretisch eine verzögerungsfreie Übertragung, da es keine Bestätigungszyklen (Acknowledge) erfordert. In der praktischen Erprobung offenbarte UDP jedoch eine kritische Instabilität: *Packet Loss*. In einem mobilen Szenario, in dem sich der Nutzer bewegt, führten verlorene Datenpakete zu korrupten JPEG-Dateien und unvollständigen

Audio-Chunks. Da UDP keine Fehlerkorrektur bietet, konnte die Applikation die defekten Binärströme nicht mehr zusammensetzen.

Phase 3: Finale hybride Topologie (BLE & TCP)
Aufgrund der Unzuverlässigkeit von UDP wechselte die Architektur final zum Transmission Control Protocol (TCP). TCP garantiert durch interne Prüfsummen und automatische Paketwiederholung eine verlustfreie Übertragung. Das System nutzt nun eine hybride Strategie: BLE übernimmt ausschließlich die Geräteerkennung und den Austausch der WLAN-Daten (Provisioning). Sobald die Verbindung steht, übernimmt ein TCP-Socket auf Port 5005 den gesamten Datenstrom. Dieser Ansatz vereint die einfache Kopplung von Bluetooth mit der Stabilität und Bandbreite eines WiFi-Netzwerks.

3.3 Interaktionen

Die Interaktion zwischen den Systemkomponenten und dem Nutzer verläuft in einem kontinuierlichen Kreislauf:

1. **Datenerfassung:** Kamera und ToF-Sensor erfassen synchron die visuelle Umgebung und Distanzdaten. Parallel überwacht die IMU physische Eingaben durch den Nutzer.
2. **Vorverarbeitung und Übertragung:** Der Mikrocontroller identifiziert relevante Objektkoordinaten und überträgt diese asynchron über eine TCP-Verbindung an das Smartphone.
3. **Fusion und Auswertung:** Das Smartphone verknüpft die Koordinaten mit Distanzwerten und wertet Sprachbefehle aus.
4. **Feedback:** Das System generiert akustische Signale oder Sprachausgaben (TTS) über das Handy.

3.4 Technisches Design

Da das System mittels Kamera und Mikrofon sensible Umgebungsdaten erfasst, ist ein striktes Sicherheitskonzept integraler Bestandteil des Entwurfs. Das Konzept verfolgt hierbei den Ansatz des *Privacy-by-Design*.

3.4.1 Lokale Datenverarbeitung und Souveränität

Im Gegensatz zu kommerziellen Smart-Glasses verzichtet „Ascenta“ konsequent auf Cloud-Schnittstellen für die Kernfunktionen. Die Objekterkennung (FOMO) findet direkt auf dem Mikrocontroller statt, während die Spracherkennung (Vosk) lokal auf dem Android-Endgerät ausgeführt wird. Dies stellt sicher, dass zu keinem Zeitpunkt Videostreams oder private Audiodaten an externe Server übertragen werden. Die Datenhoheit verbleibt vollständig auf der Hardware des Nutzers.

3.4.2 Absicherung der Netzwerkschnittstellen

Durch den Verzicht auf das verbindungslose UDP und den Einsatz des verbindungsorientierten TCP wird die Stabilität der Sitzung erhöht. Da das System in einem lokalen Netzwerk operiert, ist die Angriffsfläche minimiert. Nach dem erfolgreichen Austausch der WLAN-Credentials über BLE wird dieser Kanal für weitere Datenübertragungen geschlossen. Das System nutzt die asynchrone TCP-Struktur, um Telemetrie und Binärdaten klar voneinander zu trennen, was eine Fehlinterpretation von Datenpaketen (Cross-Talk) auf Softwareebene verhindert. Die lokale Verschlüsselung des WLAN-Access-Points sorgt zusätzlich dafür, dass Dritte den Datenstrom nicht passiv mitschneiden können.

4 Umsetzung

Das vorherige Kapitel definiert das theoretische Konzept. Dieses Kapitel beschreibt die detaillierte praktische Realisierung des Prototyps. Der Fokus liegt hierbei primär auf der komplexen softwareseitigen Implementierung der Kernfunktionen. Die Architektur vereint ein ressourcenbeschränktes Embedded-System (MicroPython auf dem Arduino Nicla Vision) mit einem asynchronen mobilen Betriebssystem (Kotlin unter Android).

4.1 Implementierung der Komponenten

4.1.1 Software-Architektur und Datenverarbeitung - Liam

Die softwareseitige Umsetzung erfordert die Koordination eines ressourcenbeschränkten Embedded-Systems (MicroPython) und eines mobilen Betriebssystems (Android). Die folgenden Abschnitte erläutern die Lösungsansätze für die Netzwerkkommunikation, die asynchrone Datenverarbeitung und das Speichermanagement detailliert.

Entwicklungsumgebungen und Werkzeuge (Toolchain)

Die softwareseitige Realisierung des „Ascenta“-Projekts erfordert den Einsatz spezialisierter Entwicklungsumgebungen (IDEs), um die unterschiedlichen Anforderungen von Embedded-Systems und mobiler App-Entwicklung effizient zu vereinen.

OpenMV IDE für MicroPython (Arduino Nicla Vision)
Für die Programmierung, das Flashen und das Debugging des Arduino Nicla Vision kommt die *OpenMV IDE* zum Einsatz [8]. Im Gegensatz zu klassischen C++ basierten Arduino-Umgebungen ist OpenMV exakt auf Machine-Vision-Anwendungen unter MicroPython zugeschnitten. Die Umgebung bietet eine direkte serielle REPL-Schnittstelle (Read-Eval-Print Loop), welche die Ausführung von Skripten in Echtzeit ohne vorherigen Kompilierungszyklus ermöglicht. Ein für die Entwicklung essenzieller Vorteil dieser IDE ist der integrierte Framebuffer-Viewer: Er überträgt das Live-Bild des Kamerasensors direkt über die USB-Schnittstelle auf den PC, was die Kalibrierung des Sichtfeldes massiv erleichtert.

Figure 4.1: Logo der OpenMV IDE

Android Studio (Kotlin Applikation)
Die Entwicklung der mobilen Applikation erfolgt in *Android Studio*, der offiziellen und von Google empfohlenen Entwicklungsumgebung für das Android-Betriebssystem [2]. Die IDE stellt umfassende Werkzeuge für die Programmierung in Kotlin bereit, darunter den Android Profiler zur Echtzeitanalyse der Netzwerk- und CPU-Auslastung.

Figure 4.2: Logo von Android Studio

Projektkonfiguration (Gradle und Manifest)

Die grundlegende Architektur, das Abhängigkeitsmanagement und die Rechteverwaltung der Android-Applikation werden über zwei zentrale Steuerungsdateien definiert.

Abhängigkeitsmanagement (build.gradle.kts)
Das *Gradle-Build-System* automatisiert den Kompilierungsprozess und verwaltet externe Bibliotheken [5]. In der `build.gradle.kts` auf App-Ebene wird die minimale Kompatibilität (`minSdk = 31`) auf Android 12 festgelegt. Dies ist eine bewusste Designentscheidung, da erst ab dieser Version die entkoppelten, modernen Bluetooth-Berechtigungen zur Verfügung stehen. Zudem werden hier die essenziellen Kernbibliotheken importiert, darunter die *Vosk Offline Engine* (`vosk-android:0.3.47`) für die lokale Spracherkennung sowie `viewBinding = true` zur typsicheren Anbindung der grafischen XML-Layouts an den Kotlin-Code.

Rechteverwaltung und Deklaration (AndroidManifest.xml)
Die Datei `AndroidManifest.xml` fungiert als Steuerzentrale gegenüber dem Betriebssystem [3]. Sie registriert die Kernkomponenten der Applikation, primär die `MainActivity` und den Netzwerk-Dienst (`AscentaService`). Ein kritischer As-

pekt ist die Deklaration der Hardware-Berechtigungen (Permissions). Da das System aktiv nach BLE-Geräten sucht und TCP-Verbindungen aufbaut, fordert das Manifest spezifische Netzwerk- und Bluetooth-Rechte an. Ausschlaggebend ist hierbei die Nutzung der neuen Android 12 Bluetooth-Rechte (`BLUETOOTH_SCAN` und `BLUETOOTH_CONNECT`), welche die feingranulare Kopplung der Brille erlauben. Um den TCP-Stream bei deaktiviertem Bildschirm aufrechtzuerhalten, wird der `AscentaService` explizit mit dem Attribut `foregroundServiceType="connectedDevice"` beim System deklariert.

Android Lifecycle-Management und Systementkopplung

Da das Smartphone während der Nutzung der Assistenzbrille typischerweise in der Tasche des Nutzers verbleibt und der Bildschirm deaktiviert ist, neigt das Android-Betriebssystem dazu, Hintergrundprozesse aus Energiespargründen zu terminieren (Doze-Mode). Die Architektur lagert daher die gesamte Netzwerk- und Bluetooth-Logik in den deklarierten `AscentaService` aus.

Durch die Eskalation dieses Dienstes zu einem *Foreground Service* priorisiert das Betriebssystem den Prozess dauerhaft auf die höchste Stufe [6]. Das Android-System verlangt für diese Priorisierung zwingend die Einbindung einer permanenten Benachrichtigung (Notification), welche dem Anwender die aktive Nutzung im Hintergrund signalisiert:

```
val notification = NotificationCompat.Builder(this, CHANNEL_ID)
    .setContentTitle("Ascenta Assistenzsystem")
    .setContentText("Netzwerkverbindung zur Brille aktiv...")
    .setSmallIcon(R.drawable.ic_eye)
    .setPriority(NotificationCompat.PRIORITY_HIGH)
    .build()

// Zwingt Android, den Service bei Display-Off nicht zu killen
startForeground(1, notification)
```

Listing 4.1: Start des Foreground Services zur Verhinderung der Terminierung

Asynchrone BLE-GATT State-Machine

Das System überträgt die WLAN-Credentials (Provisioning) initial über das Bluetooth Low Energy (BLE) Generic Attribute Profile (GATT) vom Smartphone auf die Brille. Die größte programmiertechnische Herausforderung hierbei ist die stark ereignisges-

teuerte, asynchrone API des Android-Bluetooth-Stacks [4]. Die Applikation implementiert hierfür eine strikte State-Machine innerhalb eines `BluetoothGattCallback`.

Um Puffer-Überläufe auf dem Bluetooth-Chip des Nicla Vision zu verhindern, darf das System SSID und Passwort nicht simultan senden. Die Kotlin-Logik schreibt zunächst die SSID in die entsprechende Characteristic und wartet zwingend auf den `onCharacteristicWrite`-Callback mit dem Status `GATT_SUCCESS`, bevor das Passwort kaskadierend übertragen wird:

```
override fun onCharacteristicWrite(
    g: BluetoothGatt,
    char: BluetoothGattCharacteristic,
    status: Int
) {
    if (status == BluetoothGatt.GATT_SUCCESS) {
        if (char.uuid == SSID_CHAR_UUID) {
            // SSID erfolgreich bestaetigt, nun Passwort uebermitteln
            val passChar = g.getService(PROV_SERVICE_UUID).
                getCharacteristic(PASS_CHAR_UUID)
            passChar.value = pendingPassword!!.toByteArray()
            g.writeCharacteristic(passChar)

        } else if (char.uuid == PASS_CHAR_UUID) {
            // Provisioning komplett abgeschlossen
            g.disconnect()
            startTcpConnection() // Einleitung der Hauptkommunikation
        }
    }
}
```

Listing 4.2: Kaskadierendes Provisioning der WLAN-Daten via BLE-GATT Callback

Hybrides Netzwerkprotokoll (BLE & TCP)

Nach erfolgreichem BLE-Provisioning verbindet sich der Mikrocontroller mit dem WLAN. Die App etabliert daraufhin den Hauptdatenstrom über eine verbindungsorientierte TCP-Verbindung (Port 5005). Die Architektur nutzt hierbei TCP, da dieses Protokoll durch sein internes Acknowledge-Verfahren Paketverluste bei der Übertragung großer binärer Payloads (WAV-Audio-Chunks oder JPEG-Bilder) strikt verhindert. Die Firmware konfiguriert den Socket auf dem Nicla Vision mittels `setblocking(False)`. Dies verhindert, dass Netzwerkverzögerungen den Mikrocontroller in einen Deadlock versetzen.

Stream-Multiplexing und Custom Protocol Parser

Die Nutzung einer einzigen TCP-Verbindung (Port 5005) für völlig unterschiedliche Datentypen kennzeichnet eine architektonische Besonderheit des Systems. Der Datenstrom (*Stream*) interleavt kontinuierliche Textnachrichten (JSON-Telemetrie) mit großen binären Payloads (JPEG-Bilder, WAV-Audiodaten).

Ein eigenes Applikationsprotokoll (Application Layer) verhindert eine Vermischung der Datentypen. Der Code stellt binären Übertragungen einen definierten Präfix inklusive Payload-Größe voran (z. B. `AUD_START:12500`). Der Android `AscentaService` implementiert einen maßgeschneiderten Parser auf Basis von Kotlin-Coroutines, der den Stream zunächst zeichenweise bis zum Line-Feed (`\n`) ausliest und bei Erkennung eines Binär-Präfixes temporär auf einen blockierenden Byte-Reader umschaltet:

```
tcpJob = CoroutineScope(Dispatchers.IO).launch {
    val input = tcpSocket!!.getInputStream()
    while (isActive && isConnected) {
        val line = readLineFromStream(input) // Liest ASCII bis zum Line
            -Feed \n

        if (line.startsWith("IMG_START:")) {
            val size = line.substringAfter(":").toInt()
            // Temporaerer Wechsel auf blockierenden Binär-Reader
            val imgBytes = readBytesFromStream(input, size)
            val bitmap = BitmapFactory.decodeByteArray(imgBytes, 0,
                imgBytes.size)
            // Synchronisation mit dem Main-Thread fuer UI-Updates
            withContext(Dispatchers.Main) { updateUiWithImage(bitmap) }

        } else if (line.startsWith("AUD_START:")) {
            val size = line.substringAfter(":").toInt()
            val audioBytes = readBytesFromStream(input, size)
            processAudioWithVosk(audioBytes)

        } else if (line.startsWith("{")) {
            parseTelemetryJson(line) // Uebergabe an Standard JSON-
                Parser
        }
    }
}
```

Listing 4.3: Kotlin-Coroutine für dynamisches TCP-Stream-Multiplexing und Thread-Switching

Ausfallsicherheit durch Heartbeat-Mechanismus

Die Applikation implementiert einen asynchronen Watchdog, um Verbindungsabbrüche zuverlässig zu erkennen. Über den stehenden TCP-Socket sendet die App alle zehn Sekunden einen Ping an die Brille. Bleibt der Response für mehr als 25 Sekunden aus, schließt die Software die Verbindung sicher und reaktiviert den BLE-Discovery-Modus zur automatischen Neuverbindung (*Self-Healing*):

```
heartbeatJob = CoroutineScope(Dispatchers.IO).launch {
    while (isActive) {
        if (isConnected) {
            sendCommand("ping")
            delay(10000)
            if (System.currentTimeMillis() - lastPingResponse > 25000) {
                withContext(Dispatchers.Main) {
                    isConnected = false
                    stopHeartbeat()
                    startBleScan() // Zurueck in den Discovery-Status
                }
            }
        } else { delay(5000) }
    }
}
```

Listing 4.4: Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität

Concurrency und Thread-Sicherheit in Kotlin

Die Android-Applikation führt netzwerkbasierte I/O-Operationen und rechenintensive Machine-Learning-Prozesse (Vosk) simultan aus. Dafür implementiert der Code ein striktes Thread-Dispatching mittels *Coroutines*. Sämtliche Sockets und blockierende Lesevorgänge operieren ausschließlich im Hintergrund-Thread (`Dispatchers.IO`). Sobald die Coroutine ein Datenpaket vollständig empfängt, führt sie einen synchronisierten Kontextwechsel zurück auf den Main-Thread durch (`withContext(Dispatchers.Main)`), um die Benutzeroberfläche zu aktualisieren oder die TTS-Engine konfliktfrei anzusteuern.

Ressourcenmanagement und Modell-Integration (Nicla Vision)

Die Firmware des Nicla Vision (`firmware.py`) operiert unter starken Speicherrestriktionen (ca. 1 MB nutzbarer Heap). Um Abstürze während hardwarenaher

Interrupts zu verhindern, reserviert die Firmware initial einen Notfall-Puffer über `micropython.alloc_emergency_exception_buf(100)`. Zudem erzwingt der Code eine deterministische Garbage-Collection-Strategie. Der Befehl `gc.collect()` wird strategisch nach jedem Frame-Snapshot manuell aufgerufen, um der Heap-Fragmentierung proaktiv entgegenzuwirken.

Die Kernschleife der Umgebungserfassung

Die Hauptmethode `run_detection` steuert die kontinuierliche Überwachung der Umgebung. Der Kamerasensor wird auf `sensor.RGB565` und `sensor.QVGA` konfiguriert. Ein anschließendes `windowing((240, 240))` schneidet das Bild exakt auf die benötigte Input-Tensor-Größe zu. Erkennt der Algorithmus ein valides Objekt, konstruiert die Firmware dynamisch einen JSON-String mit Label, Konfidenzwert und Distanz und streamt diesen asynchron über den TCP-Socket:

```
def run_detection():
    gc.collect() # Proaktive Defragmentierung
    try:
        sensor.set_windowing((240, 240)) # Exakter Zuschnitt auf Model-
            Input
        img = sensor.snapshot()

        # Inferenz des FOMO-Modells
        result = g_model.predict([img])[0].flatten().tolist()
        best_index = max(range(len(result)), key=lambda i: result[i])
        best_conf = result[best_index]

        label_name = LABELS[best_index] if best_index < len(LABELS) else
             "Unknown"
        distance_mm = tof.read() if tof else 0

        del img # Sofortige Freigabe des 115 KB Framebuffers
        gc.collect()

        if best_conf >= 0.6:
            json_str = json.dumps({
                "label": label_name,
                "conf": f"{best_conf:.2f}",
                "dist": distance_mm
            }) + "\n"
            send_tcp_packet(json_str.encode())

```

```
    except Exception as e:
        print(f"Vision Err: {e}")
```

Listing 4.5: Initialisierung des Sensors und Extraktion der Vorhersagewerte

Auswertung des 4D-Tensors und Sensorfusion

Die Auswertung der Vorhersagedaten des FOMO-Modells stellt eine besondere algorithmische Herausforderung dar. Das quantisierte Modell generiert einen vierdimensionalen Tensor (Heatmap), der das Bild in ein Grid unterteilt. Die Firmware iteriert über das X/Y-Grid. Findet der Algorithmus ein lokales Maximum über dem Schwellenwert, berechnet die Firmware den absoluten Bildschwerpunkt (Zentroid) und fusioniert die 2D-Bildkoordinaten mit der Z-Achse (Distanz in Millimetern) des ToF-Sensors.

Low-Level Hardware-Kommunikation (I2C & Registerauslesung)

Eine verlässliche Batterieanzeige ist für blinde Nutzer essenziell. Die Firmware liest den integrierten Fuel-Gauge-IC (MAX17262) direkt über den I2C-Bus aus. Der Mikrocontroller adressiert den Chip unter der Hexadezimal-Adresse `0x36` und fordert zwei Bytes ab dem Register `0x06` (RepSOC) an. Da der Chip die Daten hardwareseitig im *Little-Endian*-Format überträgt, konvertiert die Software die rohen Bytes in einen Integer und dividiert den Wert anschließend durch $256,0$, um den Ladezustand in Prozent zu erhalten:

```
def get_soc(self):
    if not self.fuel_gauge_i2c: return None
    try:
        # Auslesen von 2 Bytes aus dem RepSOC Register (0x06) des ICs
        data = self.fuel_gauge_i2c.readfrom_mem(0x36, 0x06, 2)

        # Konvertierung des Little-Endian Bytearrays zu einem Integer
        soc_raw = int.from_bytes(data, 'little')

        # Berechnung des realen Prozentwerts gemaess Datenblatt-Faktor
        return soc_raw / 256.0
    except:
        return None
```

Listing 4.6: Direkte I2C-Registerauslesung des State-of-Charge (SOC) vom MAX17262

Kinetische Interaktionssteuerung (Tap-Detection)

Zur Freihandbedienung (Aktivierung der Spracherkennung) liest die Firmware den internen 6-Achsen-IMU-Sensor (LSM6DSOX) via I2C aus. Um zuverlässig einen beabsichtigten Doppeltipp (Double-Tap) von der normalen Gehbewegung zu unterscheiden, wertet der Algorithmus den *Ruck* (Δg) aus.

Überschreitet der kombinierte Beschleunigungsvektor einen empirisch ermittelten Schwellenwert von $1,75$ G, speichert das System den Zeitstempel. Folgt ein zweiter Ausschlag in einem definierten Zeitfenster zwischen 100 und 600 Millisekunden, wird die Aktion als valider Double-Tap registriert und das Kommando an das Smartphone gesendet:

```
x, y, z = self.lsm.accel()
# Berechnung des Rucks (Aenderungsrate der Beschleunigung)
dx = abs(x - self.last_accel[0])
dy = abs(y - self.last_accel[1])
dz = abs(z - self.last_accel[2])
self.last_accel = (x, y, z)

delta_g = dx + dy + dz # Kombinierter Beschleunigungsvektor

if delta_g > 1.75:
    now = time.ticks_ms()
    diff = time.ticks_diff(now, self.last_tap_time)

    # Pruefung, ob der zweite Tap im validen 600ms Fenster erfolgte
    if 100 < diff < 600:
        self.send_tcp_packet(b'{"action": "tap"}\n')
        self.req_audio = True
        self.last_tap_time = 0
    elif diff > 600:
        self.last_tap_time = now # Ersten Tap als Startpunkt setzen
```

Listing 4.7: Filterung von Störbewegungen und Berechnung der Differenzbeschleunigung

Speichereffizientes Audio-Streaming

Die Aufzeichnung von Sprache stellt die extremste Belastung für den RAM dar. Eine 3,5-sekündige Audioaufnahme bei 16 kHz beansprucht exakt 112.000 Bytes. Die Firmware fragmentiert den Puffer daher in Arrays zu je 4096 Bytes. Ein hardwaregetriebener Callback füllt diese Chunks iterativ.

Durch den gezielten Einsatz von `memoryview` wird eine *Zero-Copy*-Zuweisung erzwungen: MicroPython verschiebt die Audio-Bytes direkt in die vorallozierten Speicheradressen, anstatt bei jeder Zuweisung neuen Arbeitsspeicher anzufordern. Nach Abschluss generiert die Firmware dynamisch einen 44 Byte großen RIFF/WAVE-Header mittels `struct.pack` und streamt diesen direkt mit den Daten über den TCP-Socket:

```
def create_header(data_size):
    # Bytegenaues Packing gemaess der offiziellen WAV/RIFF-Spezifikation
    return struct.pack('<4sI4s4sIHHIIHH4sI',
                       b'RIFF', 36+data_size, b'WAVE', b'fmt ', 16, 1,
                       config.CHANNELS, config.SAMPLE_RATE,
                       config.SAMPLE_RATE * config.CHANNELS * 2,
                       config.CHANNELS * 2, 16, b'data', data_size)

def _audio_callback(self, buf):
    mv = memoryview(buf)
    l = len(mv)
    off = 0
    while l > 0:
        chunk = g_chunks[self.rec_idx]
        space = AUDIO_CHUNK_SIZE - self.rec_offset
        amt = min(l, space)

        # Zero-Copy Zuweisung in den pre-allozierten Buffer
        chunk[self.rec_offset : self.rec_offset+amt] = mv[off : off+amt]
        self.rec_offset += amt
        off += amt
        l -= amt
```

Listing 4.8: Zero-Copy Speicherzuweisung und dynamische Generierung des WAV-Headers

Dynamisches Asset-Management des Akustikmodells

Ein spezifisches technisches Hindernis unter Android besteht darin, dass kompilierte C++-Bibliotheken (Vosk JNI) keine Dateien direkt aus dem komprimierten APK-Asset-Ordner lesen können. Sie erfordern absolute Dateipfade auf einem unkomprimierten Dateisystem [1]. Die Applikation löst dieses Problem durch einen asynchronen Entpackungsprozess beim Erststart. Eine IO-Coroutine iteriert rekursiv durch das Modell-Verzeichnis der Assets und kopiert die binären Akustik-Modelle Byte für Byte in den internen `filesDir`-Speicherbereich des Smartphones.

Audio-Verarbeitung und kognitive Entlastung

Nach der Transkription des Audiosignals oder der Objekterkennung erfolgt die Sprachausgabe über die offizielle `TextToSpeech`-API von Android [7]. Um den blinden Nutzer nicht mit Dauerwarnungen zu überlasten, implementiert die Applikation eine dynamische Cooldown-Logik. Befindet sich das Objekt in einer unkritischen Distanz (über 100 cm), wird ein Cooldown von sechs Sekunden für dieses spezifische Label aktiviert. Unterschreitet die vom ToF-Sensor gemessene Distanz jedoch einen kritischen Schwellenwert (z. B. unter 40 cm), greift eine Override-Logik, die den Cooldown ignoriert und eine sofortige Kollisionswarnung erzwingt.

I/O-Auslagerung beim Bild-Streaming

Die Brille bietet zur Verifikation des Sichtfeldes die Funktion, hochauflösende Snapshots an die Smartphone-App zu senden. Ein unkomprimiertes Bitmap belegt jedoch den verbleibenden RAM nahezu vollständig.

Als Lösungsarchitektur nutzt die Firmware eine *Swap-Architektur* auf dem Dateisystem. Das aufgenommene Bild wird direkt unter Anwendung einer JPEG-Kompression auf den Flash-Speicher der Brille geschrieben (`temp.jpg`). Unmittelbar danach zerstört die Firmware das rechenintensive Bildobjekt im RAM manuell durch den `del img`-Befehl und erzwingt eine Garbage-Collection. Erst mit dem nun wieder freien Arbeitsspeicher wird die Datei `temp.jpg` stückweise ausgelesen und an das Handy gestreamt:

```
def process_image(self):
    path = "temp.jpg"
    try:
        img = sensor.snapshot()
        img.save(path, quality=75) # Speichern im Flash

        del img        # Manuelle RAM-Freigabe des 115 KB Bitmaps
        gc.collect()   # Zwingende Defragmentierung

        size = os.stat(path)[6]
        self.send_tcp_packet(f"IMG_START:{size}\n".encode())

        # Speicherschonendes, stueckweises Auslesen der Swap-Datei
        with open(path, 'rb') as f:
            while True:
                chunk = f.read(1024)
                if not chunk: break
```

```
                self.send_tcp_packet(chunk)

        uos.remove(path) # Temporäre Datei sicher löschen
    except Exception as e:
        print(f"Img Err: {e}")
    finally:
        gc.collect()
```

Listing 4.9: Verwendung des Flash-Speichers als Swap zur Vermeidung von OOM-Exceptions

Benutzeroberfläche und App-Architektur (GUI als Demonstrator)

Obwohl das finale Assistenzsystem primär für blinde Nutzer über eine „Zero-UI“ konzipiert ist, spielt die grafische Benutzeroberfläche der Android-Applikation eine essenzielle Rolle als Entwickler-Dashboard und Demonstrator. Die Architektur folgt strikt dem *Single-Activity-Pattern*. Der Code unterteilt die Bildschirminhalte in modulare `Fragments`, welche die im `AscentaService` empfangenen Daten über Main-Thread-Callbacks visualisieren:

Das ImageFragment (Visueller Debugger):
Dieses Modul dient der Verifikation des Kamerablickwinkels. Sobald das Flash-Swap-Verfahren der Brille abgeschlossen ist und der TCP-Parser das `ByteArray` extrahiert hat, decodiert das Fragment die Rohdaten mittels der Android `BitmapFactory`. Da UI-Updates unter Android ausschließlich im Main-Thread erlaubt sind, nutzt das Fragment eine blockierungsfreie Coroutine-Synchronisation, um das gerenderte Bild in der `ImageView` anzuzeigen:

```
// Wird vom AscentaService aufgerufen, sobald IMG_START-Bytes gelesen
    wurden
fun updateCameraFeed(imageData: ByteArray) {
    // Wandelt die empfangenen Bytes asynchron in ein Android-Bitmap um
    val bitmap = BitmapFactory.decodeByteArray(imageData, 0, imageData.
        size)

    if (bitmap != null) {
        // UI-Updates zwingend im Main-Thread ausfuehren
        activity?.runOnUiThread {
            binding.cameraFeedImageView.setImageBitmap(bitmap)
            binding.statusText.text = "Snapshot empfangen: ${imageData.
                size} Bytes"
        }
```

```
    }
}
```

Listing 4.10: Dekodierung und Main-Thread UI-Update des empfangenen JPEG-Streams im ImageFragment

Das AudioFragment (Akustische Transkription):
Da das System Sprachbefehle lokal verarbeitet [1], ist ein visuelles Feedback der Audio-Erkennung für die Systemvalidierung unerlässlich. Wenn der Nutzer den Doppeltipp an der Brille ausführt, wird der eingehende WAV-Stream im Service an die Vosk-Engine übergeben. Das `AudioFragment` empfängt den JSON-String der Transkription und extrahiert den erkannten Text, um ihn für den Entwickler als Live-Log auf dem Bildschirm auszugeben:

```
fun onVoskResultReceived(jsonResult: String) {
    try {
        val jsonObject = JSONObject(jsonResult)
        val recognizedText = jsonObject.optString("text", "")

        if (recognizedText.isNotEmpty()) {
            activity?.runOnUiThread {
                binding.transcriptionTextView.append("\nNutzer:
                    $recognizedText")
                binding.scrollView.fullScroll(View.FOCUS_DOWN) // Auto-
                    Scroll
            }
        }
    } catch (e: Exception) {
        Log.e("AudioFragment", "Fehler beim Parsen der Vosk-Antwort", e)
    }
}
```

Listing 4.11: JSON-Parsing der Vosk-Transkription im AudioFragment

Das InfoFragment (System-Dashboard):
Dieses Modul visualisiert den globalen Systemstatus. Es zeigt in Echtzeit an, ob die TCP-Verbindung stabil ist, listet die IP-Adresse des Nicla Vision und aktualisiert dynamisch den via I2C ausgelesenen Akkustand (SOC). Ein Live-Feed im unteren Bereich listet die eingehenden Telemetriedaten auf, wodurch Entwickler sofort validieren können, welches Label mit welcher Konfidenz detektiert wurde.

4.1.2 Schaltungsdesign und Energiemanagement

Fabian

4.1.3 Entwicklung des Machine-Learning-Modells (Objekterkennung)

Fabian

4.1.4 Konstruktion und 3D-Druck

Manuel

4.2 Herausforderungen in der Entwicklung

Die Realisierung eines kompakten, tragbaren Assistenzsystems, welches rechenintensive Machine-Learning-Prozesse mit kontinuierlicher Netzwerkkommunikation vereint, bringt diverse interdisziplinäre Herausforderungen mit sich.

4.2.1 Mechanische und geometrische Restriktionen

Manuel

4.2.2 Energiemanagement im begrenzten Bauraum

Fabian

4.2.3 Ressourcenlimits und Speicher-Fragmentierung (Software) - Liam

Auf softwareseitiger Ebene stellte der extrem begrenzte Arbeitsspeicher (RAM) des Nicla Vision die größte Hürde dar. Die dynamische Speicherallokation in MicroPython führte bei der kontinuierlichen Bildverarbeitung wiederholt zur *Heap-Fragmentierung*. Benötigte das FOMO-Modell anschließend einen zusammenhängenden Speicherblock für den Tensor, schlug die Allokation trotz ausreichendem Gesamtspeicher fehl. Die stringente Einführung der *Zero-Copy*-Verfahren (`memoryview`), der Auslagerung auf

den Flash-Speicher sowie die erzwungene Garbage-Collection lösten diese Instabilitäten endgültig.

4.3 Tests und Validierung - Liam

Eine Reihe von qualitativen Praxistests belegt die Funktionstauglichkeit des „Ascenta“-Prototyps. Die Evaluierung fokussiert sich auf die Kernanforderungen eines Assistenzsystems: die wahrgenommene Systemlatenz, die Zuverlässigkeit der Sensoreingaben sowie die energetische Ausdauer auf Basis theoretischer Berechnungen und praktischer Beobachtungen.

4.3.1 Netzwerklatenz und Streaming-Performance

Da es sich bei der Objekterkennung um ein sicherheitskritisches Feature handelt, ist die Verzögerung (Latenz) zwischen physischer Hinderniserkennung und auditiver Warnung ein zentraler Bewertungsfaktor. Praxistests zeigten, dass die gewählte „Split-Brain“-Architektur den Echtzeitanforderungen gerecht wird. Der reine Telemetrie-Datentransfer (JSON) über den etablierten TCP-Stream erfolgt nahezu verzögerungsfrei, sodass Kollisionswarnungen unmittelbar generiert werden.

Der Transfer und die anschließende Offline-Transkription eines Audio-Streams durch die Vosk-Engine erfordern naturgemäß mehr Verarbeitungszeit. Dennoch erwies sich die Latenz bei der interaktiven Sprachsteuerung als ausreichend gering, um einen natürlichen Kommunikationsfluss mit dem System aufrechtzuerhalten, ohne dass es zu störenden Verzögerungen für den Nutzer kommt.

4.3.2 Auswertung der kinetischen Interaktion (Tap-Detection)

Die Zuverlässigkeit der Double-Tap-Erkennung über die IMU (LSM6DSOX) wurde in verschiedenen Bewegungsszenarien evaluiert. Dabei wurden bewusste Eingaben (Taps) den typischen Störbewegungen eines Nutzers (wie schnelles Gehen, Kopfdrehungen oder Erschütterungen) gegenübergestellt.

Die Auswertung ergab, dass der im Code definierte Schwellenwert der Differenzbeschleunigung ($\Delta g > 1,75$) eine effektive Balance bietet. Normale Kopfbewegungen und Erschütterungen beim Gehen werden adäquat herausgefiltert, wodurch Fehlalarme

(False-Positives) stark minimiert werden. Gleichzeitig registriert das System beabsichtigte Doppeltipps zuverlässig, ohne dem Anwender eine unverhältnismäßig hohe Schlagkraft auf das Brillengestell abzuverlangen.

4.3.3 Energetische Evaluierung (Akkulaufzeit)

Da standardisierte Messungen unter exakt reproduzierbaren Laborbedingungen im Rahmen dieses Prototyps nicht realisierbar waren, stützt sich die energetische Evaluierung auf eine theoretische Berechnung gemäß den offiziellen Datenblattwerten des Herstellers.

Bei einer verbauten Gesamtkapazität von 1240 mAh (durch den Parallelbetrieb von zwei 620 mAh LiPo-Zellen) ergeben sich folgende theoretische Rechenwerte: Für den aktiven Betrieb (Kamera eingeschaltet) spezifiziert das Datenblatt des Arduino Nicla Vision einen durchschnittlichen Stromverbrauch von 105 mA. Unter dieser Last errechnet sich eine theoretische Maximallaufzeit von exakt $11,8$ Stunden (1240 mAh/105 mA).

Bei diesen 105 mA handelt es sich jedoch um einen isolierten Idealwert des reinen Mikrocontrollers. Die reale Anwendung im Assistenzsystem schließt zwingend weitere energiefordernde Faktoren mit ein: Die kontinuierliche Funkübertragung im lokalen WiFi-Netzwerk (TCP-Stream) sowie die simultane Auslesung zusätzlicher Sensorik (Time-of-Flight-Laser und IMU). Da das System keinen energiesparenden Idle-Modus besitzt und permanent unter diesen Bedingungen arbeitet, wird die theoretische Akkulaufzeit von knapp zwölf Stunden in der Praxis nicht erreicht. Dennoch bestätigen die qualitativen Feldtests, dass die verbleibende reale Laufzeit den geplanten Alltagseinsatz – beispielsweise während eines mehrstündigen Einkaufs – problemlos und ausfallsicher abdeckt.

5 Ausblick (Future Work)

Der aktuelle Prototyp des Assistenzsystems „Ascenta“ belegt die Machbarkeit einer dezentralen, offline-fähigen Smart-Glasses-Architektur. Um das System jedoch von einem funktionalen Prototyp zu einem marktreifen Endkundenprodukt weiterzuentwickeln, sind in zukünftigen Iterationen verschiedene Erweiterungen und Optimierungen in allen Fachbereichen erforderlich.

5.1 Software und Benutzeroberfläche - Liam

Die zukünftige Software-Entwicklung muss sich primär auf die vollständige Barrierefreiheit und erweiterte Navigationskonzepte konzentrieren.

5.1.1 Native Zero-UI-Applikation für blinde Nutzer

Die aktuelle Android-Applikation verfügt über eine grafische Benutzeroberfläche (GUI), die essenziell für das Debugging, die Evaluierung und die Ersteinrichtung durch sehende Begleitpersonen ist. In einer finalen Version für den Endanwender muss diese App jedoch als reine Hintergrundanwendung (*Zero-UI*) konzipiert werden.

Das visuelle Interface wird dabei vollständig obsolet. Die Applikation sollte sich nahtlos in native Accessibility-Dienste (wie Android TalkBack) integrieren und den Setup-Prozess (z. B. die Eingabe des WLAN-Passworts) über auditive Menüführung oder NFC-Tags abwickeln. Der Fokus liegt dabei auf maximaler Stabilität des Foreground-Services und einer Reduktion des Batterie- und Speicherverbrauchs des Smartphones, da keine visuellen Elemente gerendert werden müssen.

5.1.2 Kontinuierliches auditives Raum-Feedback (Leitstrahl-Prinzip)

Eine vielversprechende funktionale Erweiterung stellt die Implementierung eines dynamischen, akustischen Suchmodus dar. Sucht der Nutzer nach einem bestimmten Objekt (z. B. einem Stuhl oder einer Tür), könnte das System ein kontinuierliches Au-

diosignal generieren, das an die Funktion eines Geigerzählers oder eines Einparkhilfesensors angelehnt ist.

Wenn der Nutzer den Kopf schwenkt und das gesuchte Objekt in das Sichtfeld der Kamera gerät, passt der Algorithmus die Frequenz (Tonhöhe) und das Intervall (Pings pro Sekunde) des Audiosignals in Echtzeit an. Je zentrierter das Objekt im Sichtfeld ist und je geringer die Distanz (gemessen durch den Time-of-Flight-Sensor) wird, desto intensiver wird das akustische Feedback. Dies würde blinden Nutzern eine hochpräzise, intuitive Zielführung ermöglichen, ohne sie mit komplexen, gesprochenen Sätzen zu überlasten.

5.2 Künstliche Intelligenz und Modell-Training

5.2.1 Erweiterte Objekterkennung für Alltagsgegenstände

Derzeit ist das System auf die Erkennung von Personen optimiert. Für eine umfassende Alltagshilfe muss das zugrundeliegende Dataset massiv erweitert werden. Die Firmware und die App-Logik sind bereits so abstrahiert, dass sie beliebige Labels verarbeiten können. Zukünftige Iterationen müssen daher trainiert werden, um eine Vielzahl von Hindernissen und Zielen (z. B. Treppenstufen, Ampeln, Zebrastreifen, Türen, Sitzgelegenheiten oder herannahende Fahrzeuge) zuverlässig zu klassifizieren.

5.2.2 Weiterentwicklung des Machine-Learning-Modells (Edge Impulse)

Fabian

5.3 Hardware und Elektronik

5.3.1 Optimierung der Schaltungstechnik und Sensorik

Fabian

5.4 Mechanik und Gehäusedesign

5.4.1 Weiterentwicklung des 3D-Modells und der Ergonomie

Manuel

6 Verwandte Arbeiten (Related Work) - Liam

Die Entwicklung von Assistenzsystemen für sehbeeinträchtigte Personen ist ein etabliertes Forschungs- und Entwicklungsfeld. Um die Positionierung und den Mehrwert des „Ascenta“-Projekts zu verdeutlichen, werden in diesem Kapitel bestehende Lösungsansätze analysiert und dem hier entwickelten System gegenübergestellt. Die existierenden Systeme lassen sich grob in klassische Hilfsmittel, softwarebasierte Smartphone-Lösungen und dedizierte, kommerzielle Wearables unterteilen.

6.1 Klassische Mobilitätshilfen

Das mit Abstand am weitesten verbreitete Hilfsmittel ist der klassische weiße Langstock. Er ermöglicht durch taktiles Abtasten die Erkennung von Bodenhindernissen, Treppenstufen und Bordsteinkanten. **Abgrenzung:** Der Langstock ist eine essenzielle Basisversorgung, weist jedoch einen entscheidenden blinden Fleck auf: Hindernisse auf Rumpf- und Kopfhöhe (z. B. offene Fenster, Verkehrsschilder oder Äste) werden nicht erfasst, was ein erhebliches Verletzungsrisiko birgt. „Ascenta“ versteht sich nicht als Ersatz für den Langstock, sondern als komplementäres System, welches genau diesen toten Winkel durch den Einsatz des Time-of-Flight-Sensors (ToF) abdeckt.

6.2 Smartphone-basierte Applikationen

In den letzten Jahren haben sich Smartphone-Apps etabliert, die die Handykamera zur Umgebungsanalyse nutzen. Prominente Beispiele sind *Seeing AI* (Microsoft) oder *Be My Eyes*. Diese Applikationen können Texte vorlesen, Währungen erkennen oder Videotelefonate mit sehenden Freiwilligen aufbauen. **Abgrenzung:** Der größte Nachteil dieser Lösungen ist mangelnde Ergonomie. Der Nutzer muss das Smartphone permanent in der Hand halten und aktiv auf Objekte richten, was die Hände für die Nutzung des Langstocks oder von Blindenführhunden blockiert. Zudem basieren diese Apps fast ausschließlich auf Cloud-Computing. Dies erfordert eine konstante Breitband-Internetverbindung und wirft datenschutzrechtliche Bedenken auf,

da sensible Videoströme der Umgebung an externe Server gesendet werden. „Ascenta“ löst dieses Problem durch den „Hands-free“-Wearable-Ansatz und die strikte, lokale Datenverarbeitung (Offline-Edge-AI).

6.3 Kommerzielle Smart Glasses und Wearables

Auf dem kommerziellen Markt existieren hochpreisige Speziallösungen. Das bekannteste System ist *OrCam MyEye*, ein kompaktes Kameramodul, das magnetisch an herkömmlichen Brillen befestigt wird. Es liest Texte vor und erkennt Gesichter. Ein weiteres Beispiel sind die *Envision Glasses*, welche auf der Google Glass Enterprise Edition 2 basieren und komplexe Szenenbeschreibungen mithilfe von Künstlicher Intelligenz liefern. **Abgrenzung:** Obwohl diese Systeme technologisch hochentwickelt sind, weisen sie massive Hürden in der Zugänglichkeit auf. Die Anschaffungskosten liegen typischerweise zwischen 2.500 und 4.000 Euro, was sie für einen Großteil der Betroffenen – insbesondere in Entwicklungsländern oder ohne vollständige Kostenübernahme durch Krankenkassen – unerschwinglich macht.

6.4 Positionierung des von Ascenta

Das Projekt „Ascenta“ positioniert sich als Brückenschlag zwischen kostengünstiger Machbarkeit und datenschutzkonformer Hochtechnologie. Durch die „Split-Brain“-Architektur wird die teure und wärmeintensive Rechenhardware (wie sie in den Envision Glasses verbaut ist) durch das ohnehin beim Nutzer vorhandene Smartphone ersetzt.

Im Gegensatz zu den kommerziellen Systemen belaufen sich die Materialkosten für den Ascenta-Prototyp (Arduino Nicla Vision, LiPo-Akkus, 3D-Druck-Filament) auf unter 150 Euro. Gleichzeitig garantiert die lokale Ausführung des FOMO-Modells sowie der Vosk-Spracherkennungs-Engine eine 100-prozentige Offline-Funktionalität, was die Reaktionszeiten (Latenz) minimiert und die Privatsphäre des Nutzers und dessen sozialen Umfelds absolut schützt.

7 Zusammenfassung und Ausblick - Liam

8 Project Management

8.1 Planung und Organisation

Die organisatorische Planung und Umsetzung des Projekts „Ascenta“ basierte auf einer meilensteinorientierten, agilen Vorgehensweise. Zu Beginn des Projekts (Vorbereitungsphase) lag der Fokus auf der grundlegenden organisatorischen Planung, der Definition von Grob- und Feinzielen sowie der Recherche und Beschaffung der elektronischen Komponenten.

Die formelle Strukturierung der Arbeitsschritte und die Einreichung der Projektmeilensteine erfolgten über das schulspezifische ABA-Portal. Um eine strukturierte Arbeitsteilung zwischen den Bereichen Software, Elektronik und 3D-Druck zu gewährleisten, wurde der Projektplan in wöchentliche Sprints (Kalenderwochen, KW) unterteilt. Zur Überwachung des Fortschritts und der individuellen Arbeitslast wurde eine detaillierte Zeiterfassung (Timesheet) mittels Tabellenkalkulation implementiert, welche die geplanten Freizeitstunden dem tatsächlichen Aufwand (Soll/Ist-Vergleich) kontinuierlich gegenüberstellt.

8.2 Evaluierung der Arbeitsstunden

Die wöchentliche Evaluierung der Tätigkeitsberichte ermöglichte eine präzise Überwachung des Projektfortschritts. Ein Abgleich der geplanten Ressourcen mit dem tatsächlichen Aufwand zeigt insbesondere in der initialen Projektphase eine deutliche Diskrepanz: Während für die Projektdefinition anfänglich 6 Stunden pro Teammitglied veranschlagt wurden, belief sich der tatsächliche Aufwand auf 20 Stunden pro Person. Diese anfängliche Unterschätzung ist typisch für komplexe Hardware-Software-Kombinationen, da grundlegende Architektur-Entscheidungen zeitintensiv sind.

In den darauffolgenden Wochen (beispielsweise in KW 37 und 38) stabilisierte sich die Arbeitslast und entsprach mit durchschnittlich 6 Stunden pro Teammitglied exakt den Planwerten. Die konsequente Dokumentation der wöchentlichen Ist-Zustände – wie etwa der erfolgreiche Test des Spannungsbetriebsbereichs der Akkus oder die Evaluation der Micro-Speech-Software – half maßgeblich dabei, technische Hürden frühzeitig

zu identifizieren und den Projektplan dynamisch anzupassen.

8.3 Zeitplan

Die nachfolgenden Unterschriften bestätigen die Korrektheit der in der internen Zeiterfassung dokumentierten, individuell erbrachten Arbeitsstunden für die Diplomarbeit.

8.3.1 Liam Vendel

Braunau am Inn, March 18, 2026 Liam Vendel
Ort, Datum

Gesamtarbeitsaufwand: [XX] Stunden

8.3.2 Manuel Zauner

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 18, 2026 Manuel Zauner _____
Ort, Datum *Unterschrift*

8.3.3 Fabian Lindlbauer

Gesamtarbeitsaufwand: [XX] Stunden

Braunau am Inn, March 18, 2026	Fabian Lindlbauer	_____
Ort, Datum		*Unterschrift*

Acknowledgements

List of Figures

4.1 Logo der OpenMV IDE 11
4.2 Logo von Android Studio 11

Quellcodeverzeichnis

3.1 Berechnung des Audiospeichers und initiales BLE-Setup 7

4.1 Start des Foreground Services zur Verhinderung der Terminierung . . . 12
4.2 Kaskadierendes Provisioning der WLAN-Daten via BLE-GATT Callback 13
4.3 Kotlin-Coroutine für dynamisches TCP-Stream-Multiplexing und Thread-Switching 14
4.4 Heartbeat-Coroutine zur Überwachung der Verbindungsstabilität 15
4.5 Initialisierung des Sensors und Extraktion der Vorhersagewerte 16
4.6 Direkte I2C-Registerauslesung des State-of-Charge (SOC) vom MAX17262 17
4.7 Filterung von Störbewegungen und Berechnung der Differenzbeschleunigung 18
4.8 Zero-Copy Speicherzuweisung und dynamische Generierung des WAV-Headers 19
4.9 Verwendung des Flash-Speichers als Swap zur Vermeidung von OOM-Exceptions 20
4.10 Dekodierung und Main-Thread UI-Update des empfangenen JPEG-Streams im ImageFragment 21
4.11 JSON-Parsing der Vosk-Transkription im AudioFragment 22

Literaturverzeichnis

[1] Alpha Cephei Inc. Vosk Offline Speech Recognition API. https://alphacephei.com/vosk/. Zugegriffen: 12.03.2026.

[2] Google LLC. Android Studio and Gradle Build System. https://developer.android.com/studio. Zugegriffen: 12.03.2026.

[3] Google LLC. App Manifest Overview | Android Developers. https://developer.android.com/guide/topics/manifest/manifest-intro. Zugegriffen: 12.03.2026.

[4] Google LLC. BluetoothGatt API Reference | Android Developers. https://developer.android.com/reference/android/bluetooth/BluetoothGatt. Zugegriffen: 12.03.2026.

[5] Google LLC. Configure your build | Android Studio. https://developer.android.com/studio/build. Zugegriffen: 12.03.2026.

[6] Google LLC. Foreground Services | Android Developers. https://developer.android.com/guide/components/foreground-services. Zugegriffen: 12.03.2026.

[7] Google LLC. TextToSpeech | Android Developers. https://developer.android.com/reference/android/speech/tts/TextToSpeech. Zugegriffen: 12.03.2026.

[8] OpenMV LLC. OpenMV IDE and MicroPython Documentation. https://docs.openmv.io/. Zugegriffen: 12.03.2026.

LIAM VENDEL

Gampern, Österreich | +43 664 1851637 |
Liamvendel06@gmail.com
github.com/schlaimwendla

PROFIL

Angehender Elektronik-Absolvent der HTBLA Braunau mit dem Schwerpunkt **Coding & AI**. Spezialisiert auf Backend Engineering und verteilte Systeme. Fundierte Kenntnisse in Java (OOP, Concurrency) und Low-Level Architektur. Begeisterter Linux-Nutzer (Arch/Garuda) mit Fokus auf Clean Code und Performance.

TECHNISCHE SKILLS

Programmierung:	Java (Fortgeschritten), Python, C++, MicroPython, Assembly (Memory Management).
Systeme & Tools:	Linux (CLI Experte), Docker, Git, Spring Boot, REST APIs, Bash Scripting.
KI & Hardware:	Nicla Vision, ESP32, Raspberry Pi, Edge Impulse, Wit.ai (NLP).
Sprachen:	Deutsch (Muttersprache), Englisch (Fließend).

DIPLOMARBEIT: "ASCENTA"

Smarte Assistenzbrille mit Sprachsteuerung *seit 2025*

- Entwicklung einer synchronisierten **UDP/BLE-Kommunikationsstruktur** zwischen Wearable (Nicla Vision) und Mobile App (Android/Kotlin).
- Integration von **Edge AI** zur Latenzoptimierung (Objekterkennung via Edge Impulse).
- Implementierung einer Offline-Sprachsteuerung zur Gewährleistung maximaler Privatsphäre.

PROJEKTE & HIGHLIGHTS

IoT Light Control System *Raspberry Pi / Python*

- Entwicklung eines REST-API Backends zur modularen und drahtlosen Steuerung von Lichtmodulen.

Automatic Guitar Tuner *Arduino / Signalverarbeitung*

- Design eines elektromechanischen Stimmgeräts auf Basis von Frequenzanalyse und Feedback-Loops.

European Space Camp 2025 *Andøya Space Center, Norwegen*

- Auswahl für das internationale Forschungs-Camp (Fokus: Sounding Rockets & Systems Engineering).

AUSBILDUNG

HTBLA Braunau **2021 – 2026**

Höhere Technische Bundeslehranstalt für Elektronik und Technische Informatik
Schwerpunkt: **Coding & AI** (Voraussichtlicher Abschluss: Juni 2026)

INTERESSEN & ENGAGEMENT

- **Open Source:** Linux-Kernel & System-Optimierung (Btrfs-Subvolumes, Kernel-Tuning).
- **Sport:** Aktiver Volleyball-Spieler und Jugend-Coach im Verein.
- **Raumfahrt:** NASA Extracurricular Kurse in Weltraumphysik und orbitaler Mechanik.

Manuel Zauner

Höfter Straße 70a

5280 Braunau am Inn

Tel: 0677 62550900

E-Mail: manuel.zauner.2000@gmail.com

Lebenslauf

Angaben zur Person

Name:	Manuel Zauner
Geburtsdatum:	19.01.2007
Geburtsort:	Braunau am Inn
Staatsbürgerschaft:	Österreich

Familiensituation:

Vater:	Christian Zauner (Technischer Angestellter, Fa. Spiratec)
Mutter:	Bettina Zauner (Personalverrechnung, Fa. EKB)
Geschwister:	Thomas Zauner (Gymnasium Braunau)

Schulbildung

Art/Dauer der Schule:	2013 – 2017	VS Braunau-Laab
	2017 – 2021	Gymnasium Braunau
	2021 – laufend	HTL Braunau (Coding & AI)
Lieblingsfach:	PBE_3 (Prototypenbau elektronischer Systeme)	

Berufserfahrung

Ferialjob:	August 2023	4 Wochen in der Firma Dräxlmaier
	August 2024	4 Wochen in der Firma Dräxlmaier

Sonstige Angaben

Besondere Fähigkeiten:	EDV, gute Menschenkenntnisse
Hobbys:	Basketball, Krafttraining, Tennis, Computer

Lebenslauf

Name: Fabian Lindlbauer
Geburtsdaten: 24.07.2007
Anschrift: Moritzhub 8, 4950 Altheim
Telefon: 06504307493
E-Mail: fabian.lindlbauer007@gmail.com

Schulausbildung:

2013-2017	Volksschule Altheim
2017-2021	Mittelsschule Altheim
2021-	HTL Braunau

Berufserfahrung:

Juli-Juli 2023	Hargassner Weng	Ferialpraktikum
Juli-Juli 2024	SGS Dorf an der Pram	Ferialpraktikum
Juli-Juli 2025	HTL Braunau Schulpartnerschaft Nicaragua	

Fremdsprache:

Englisch B2

ECDL
Führeschein B

Altheim, 11.09.2025

Fabian Lindllbauer